

Homework 5

```
# Chapter 10, Question 7
data("Default")
set.seed(1)
n <- nrow(Default)
train_idx <- sample(n, 0.7*n)
train <- Default[train_idx, ]
test <- Default[-train_idx, ]

# Logistic regression
glm_fit <- glm(default ~ ., data = train, family = binomial)
glm_prob <- predict(glm_fit, newdata = test, type = "response")
glm_pred <- ifelse(glm_prob > 0.5, "Yes", "No")
glm_acc <- mean(glm_pred == test$default)

# Neural network with dropout
# Convert student to numeric and default to 0/1
train_x <- cbind(train[, c("balance", "income")],
                 student = as.numeric(train$student == "Yes"))
train_y <- as.numeric(train$default == "Yes")
test_x <- cbind(test[, c("balance", "income")],
               student = as.numeric(test$student == "Yes"))
test_y <- as.numeric(test$default == "Yes")

# Scale features
mean_x <- apply(train_x, 2, mean)
sd_x <- apply(train_x, 2, sd)
train_x_scaled <- scale(train_x, center = mean_x, scale = sd_x)
test_x_scaled <- scale(test_x, center = mean_x, scale = sd_x)

model <- keras_model_sequential() %>%
  layer_dense(units = 10, activation = "relu", input_shape = ncol(train_x_scaled)) %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 1, activation = "sigmoid")

model %>% compile(
  optimizer = "adam",
  loss = "binary_crossentropy",
  metrics = c("accuracy")
)

history <- model %>% fit(
  train_x_scaled, train_y,
  epochs = 50,
  batch_size = 32,
  validation_split = 0.2,
  verbose = 0
)
```

```

)

nn_prob <- model %>% predict(test_x_scaled)

## 94/94 - 0s - 4ms/step

nn_pred <- ifelse(nn_prob > 0.5, 1, 0)
nn_acc <- mean(nn_pred == test_y)

cat("Logistic regression accuracy:", glm_acc, "\n")

## Logistic regression accuracy: 0.9723333

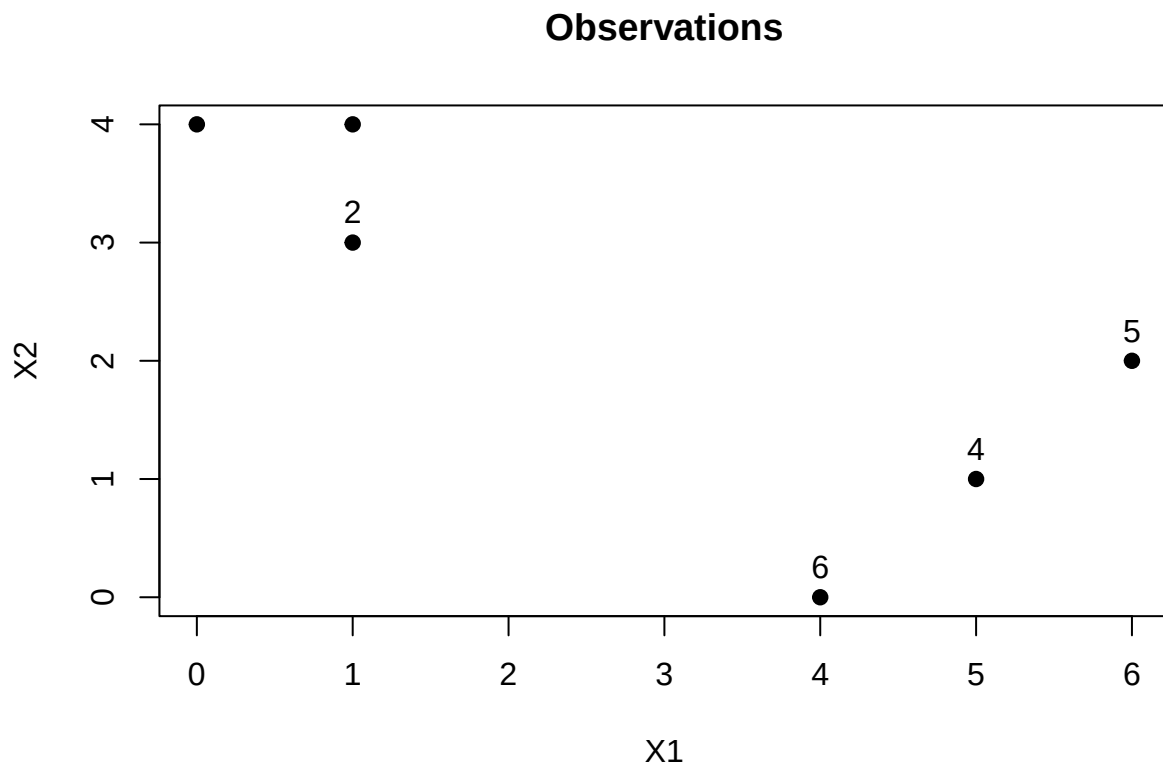
cat("Neural network accuracy:", nn_acc, "\n")

## Neural network accuracy: 0.966

obs <- matrix(c(1,4, 1,3, 0,4, 5,1, 6,2, 4,0), ncol = 2, byrow = TRUE)
colnames(obs) <- c("X1", "X2")
rownames(obs) <- 1:6

plot(obs, pch = 19, xlab = "X1", ylab = "X2", main = "Observations")
text(obs, labels = rownames(obs), pos = 3)

```



```
# (b) random assignment
set.seed(2)
cluster <- sample(1:2, 6, replace = TRUE)
cat("Initial cluster labels:\n")
```

```
## Initial cluster labels:
```

```
print(cluster)
```

```
## [1] 1 1 2 2 2 2
```

```
# (c) compute centroids
centroids <- matrix(NA, nrow = 2, ncol = 2)
for(k in 1:2) {
  if(sum(cluster == k) > 0) {
    centroids[k, ] <- colMeans(obs[cluster == k, , drop = FALSE])
  } else {
    centroids[k, ] <- obs[sample(1:6, 1), ] # random if empty
  }
}
cat("Initial centroids:\n")
```

```
## Initial centroids:
```

```
print(centroids)
```

```
##      [,1] [,2]
## [1,] 1.00 3.50
## [2,] 3.75 1.75
```

```
# (d) assign to nearest centroid
assign_clusters <- function(obs, centroids) {
  dists <- matrix(NA, nrow = nrow(obs), ncol = nrow(centroids))
  for(i in 1:nrow(obs)) {
    for(k in 1:nrow(centroids)) {
      dists[i, k] <- sqrt(sum((obs[i, ] - centroids[k, ])^2))
    }
  }
  return(apply(dists, 1, which.min))
}
```

```
new_cluster <- assign_clusters(obs, centroids)
cat("Clusters after first assignment:\n")
```

```
## Clusters after first assignment:
```

```
print(new_cluster)
```

```
## [1] 1 1 1 2 2 2
```

```
# (e) repeat until convergence
```

```
repeat {  
  old <- new_cluster  
  centroids <- matrix(NA, nrow = 2, ncol = 2)  
  for(k in 1:2) {  
    if(sum(new_cluster == k) > 0) {  
      centroids[k, ] <- colMeans(obs[new_cluster == k, , drop = FALSE])  
    } else {  
      centroids[k, ] <- obs[sample(1:6, 1), ]  
    }  
  }  
  new_cluster <- assign_clusters(obs, centroids)  
  if(all(new_cluster == old)) break  
}  
cat("Final cluster labels:\n")
```

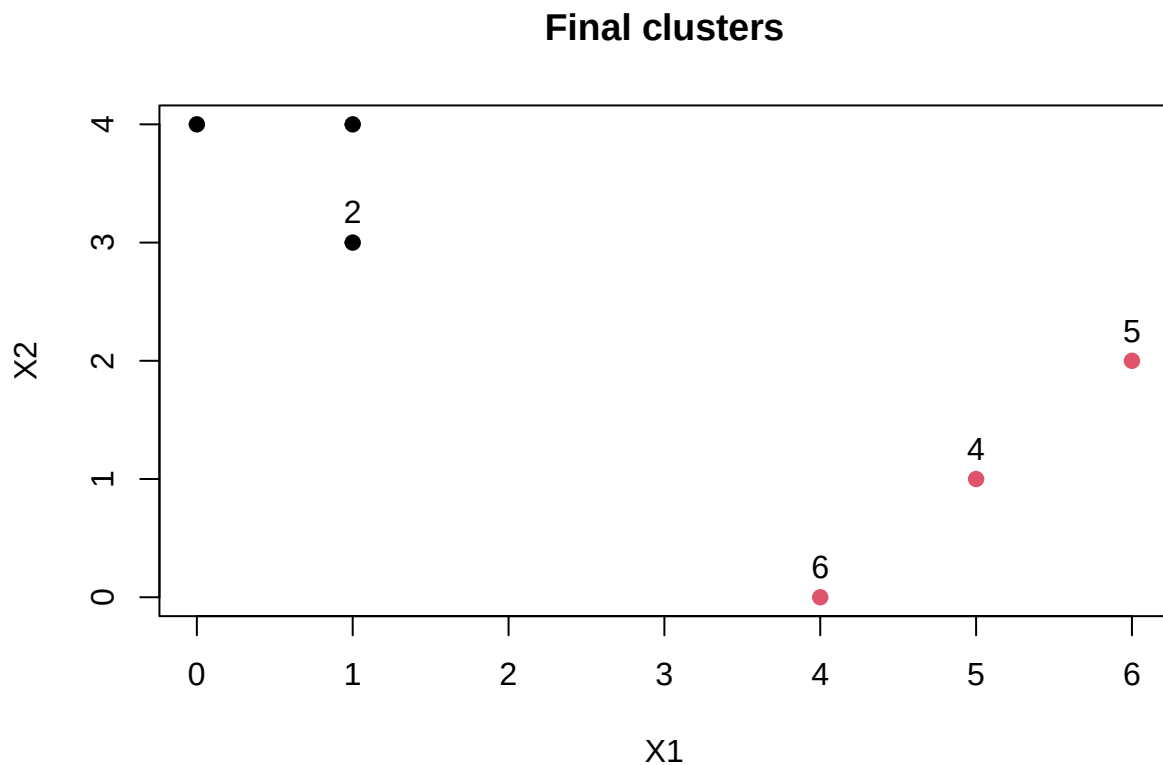
```
## Final cluster labels:
```

```
print(new_cluster)
```

```
## [1] 1 1 1 2 2 2
```

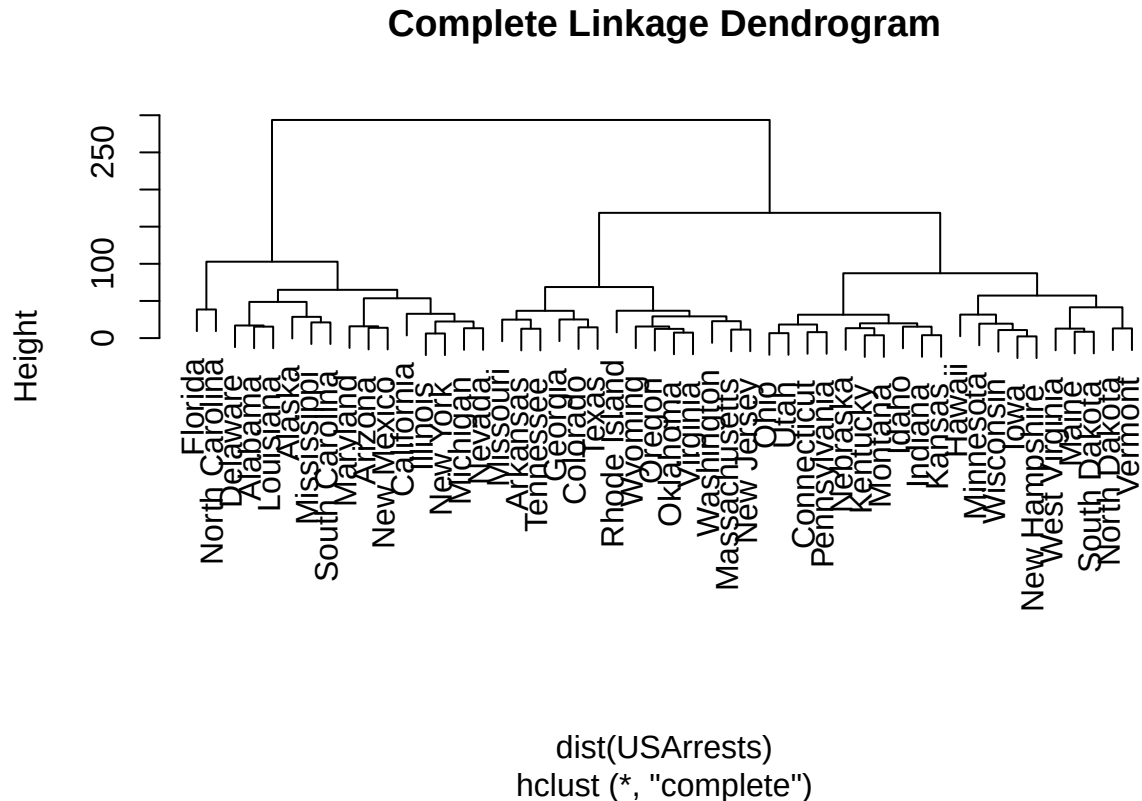
```
# (f) plot colored
```

```
plot(obs, pch = 19, col = new_cluster, xlab = "X1", ylab = "X2", main = "Final clusters")  
text(obs, labels = rownames(obs), pos = 3)
```



```
data("USArrests")

# (a) hierarchical clustering complete linkage Euclidean
hc_complete <- hclust(dist(USArrests), method = "complete")
plot(hc_complete, main = "Complete Linkage Dendrogram")
```



```
# (b) cut at height for 3 clusters
clusters3 <- cutree(hc_complete, k = 3)
cat("Clusters (complete linkage, no scaling):\n")
```

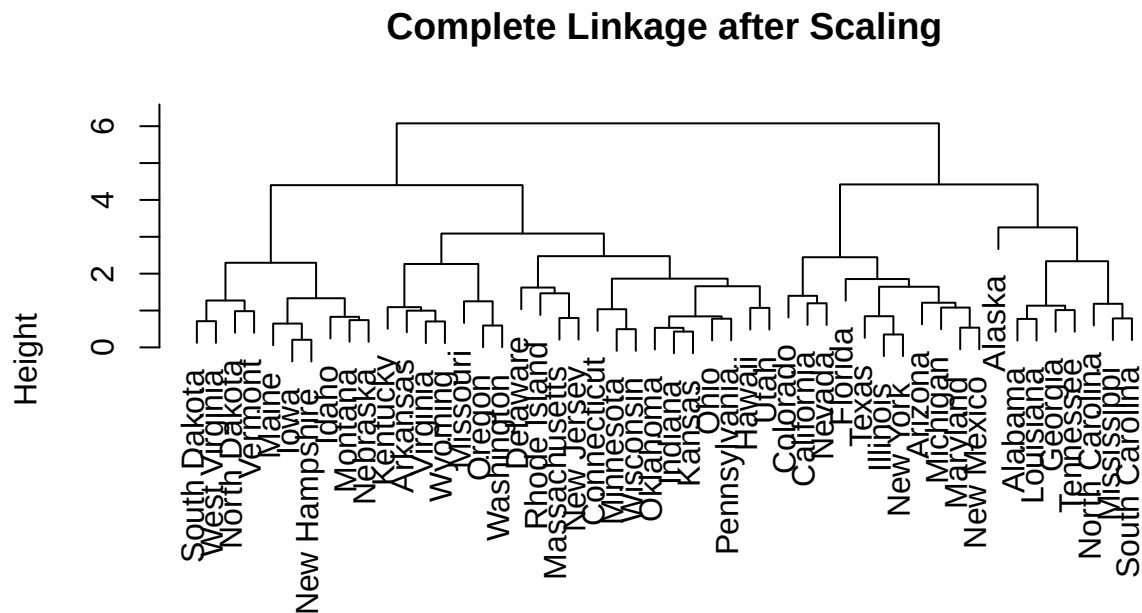
```
## Clusters (complete linkage, no scaling):
```

```
print(clusters3)
```

```
##      Alabama      Alaska      Arizona      Arkansas      California
##           1           1           1           2           1
##   Colorado Connecticut Delaware      Florida      Georgia
##           2           3           1           1           2
##      Hawaii      Idaho      Illinois      Indiana      Iowa
##           3           3           1           3           3
##      Kansas      Kentucky Louisiana      Maine      Maryland
##           3           3           1           3           1
## Massachusetts Michigan Minnesota Massachusetts Missouri
##           2           1           3           1           2
```

```
##      Montana      Nebraska      Nevada  New Hampshire  New Jersey
##      3            3            1            3            2
##      New Mexico    New York  North Carolina  North Dakota      Ohio
##      1            1            1            3            3
##      Oklahoma      Oregon    Pennsylvania    Rhode Island  South Carolina
##      2            2            3            2            1
##      South Dakota  Tennessee      Texas            Utah            Vermont
##      3            2            2            3            3
##      Virginia      Washington  West Virginia    Wisconsin      Wyoming
##      2            2            3            3            2
```

```
# (c) scale variables and cluster
USArrests_scaled <- scale(USArrests)
hc_scaled <- hclust(dist(USArrests_scaled), method = "complete")
plot(hc_scaled, main = "Complete Linkage after Scaling")
```



```
dist(USArrests_scaled)
hclust(*, "complete")
```

```
clusters3_scaled <- cutree(hc_scaled, k = 3)
cat("Clusters after scaling:\n")
```

```
## Clusters after scaling:
```

```
print(clusters3_scaled)
```

```
##      Alabama      Alaska      Arizona      Arkansas      California
```

##	1	1	2	3	2
##	Colorado	Connecticut	Delaware	Florida	Georgia
##	2	3	3	2	1
##	Hawaii	Idaho	Illinois	Indiana	Iowa
##	3	3	2	3	3
##	Kansas	Kentucky	Louisiana	Maine	Maryland
##	3	3	1	3	2
##	Massachusetts	Michigan	Minnesota	Mississippi	Missouri
##	3	2	3	1	3
##	Montana	Nebraska	Nevada	New Hampshire	New Jersey
##	3	3	2	3	3
##	New Mexico	New York	North Carolina	North Dakota	Ohio
##	2	2	1	3	3
##	Oklahoma	Oregon	Pennsylvania	Rhode Island	South Carolina
##	3	3	3	3	1
##	South Dakota	Tennessee	Texas	Utah	Vermont
##	3	1	2	3	3
##	Virginia	Washington	West Virginia	Wisconsin	Wyoming
##	3	3	3	3	3

(d) effect of scaling

Explanation: Scaling gives each variable equal weight; without scaling, variables with larger variance

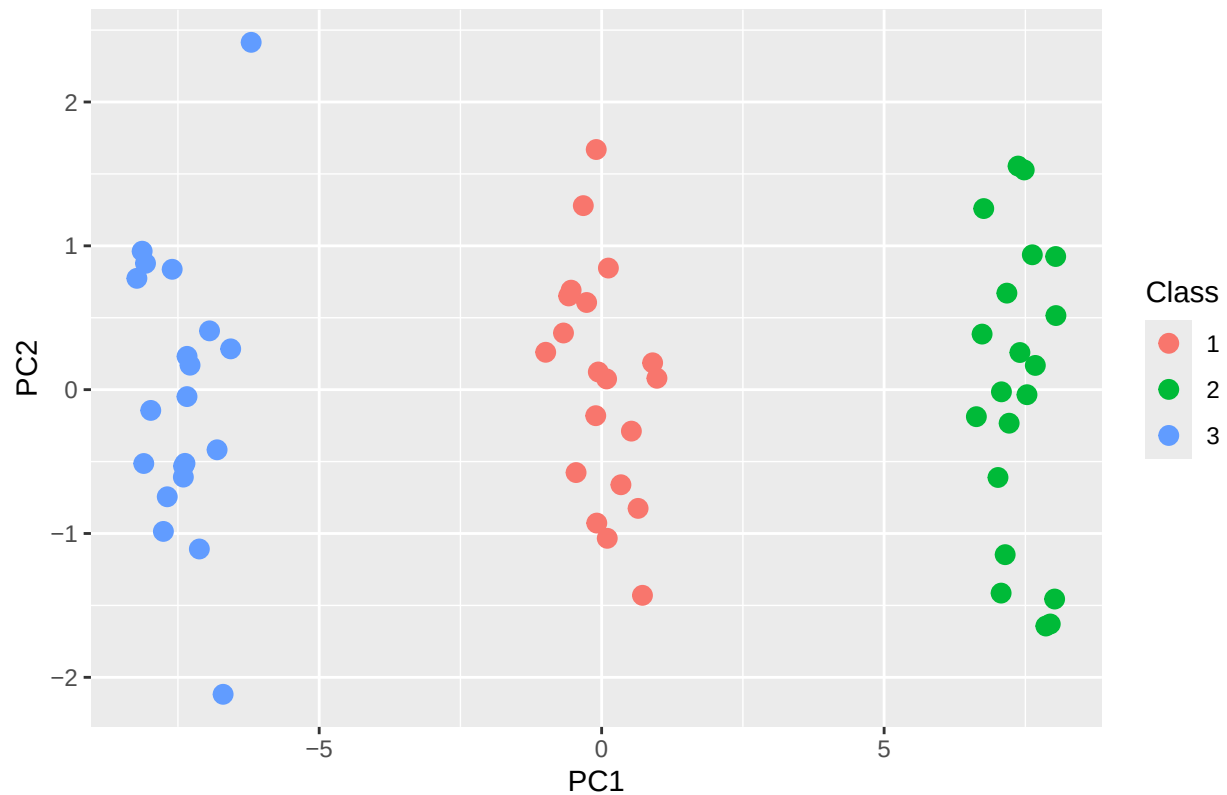
In my opinion, scaling is advisable when variables are measured on different scales to avoid one variable

```
set.seed(123)
n <- 60
p <- 50
# Three classes, 20 each
class_labels <- rep(1:3, each = 20)
# Generate data: each variable iid N(0,1) plus class mean
mu <- matrix(0, nrow = 3, ncol = p)
mu[1, ] <- 0
mu[2, ] <- 2
mu[3, ] <- -2
X <- matrix(NA, nrow = n, ncol = p)
for(i in 1:n) {
  class <- class_labels[i]
  X[i, ] <- rnorm(p, mean = mu[class, ], sd = 1)
}
```

(a) Already generated

```
# (b) PCA
pca_out <- prcomp(X, scale = TRUE)
pca_scores <- pca_out$x
pca_df <- data.frame(PC1 = pca_scores[,1], PC2 = pca_scores[,2], Class = factor(class_labels))
ggplot(pca_df, aes(x = PC1, y = PC2, color = Class)) +
  geom_point(size = 3) +
  ggtitle("PCA of simulated data")
```

PCA of simulated data



```
# (c) K-means K=3
km3 <- kmeans(X, centers = 3, nstart = 20)
table(true = class_labels, cluster = km3$cluster)
```

```
##      cluster
## true  1  2  3
##    1  0  0 20
##    2  0 20  0
##    3 20  0  0
```

```
# (d) K=2
km2 <- kmeans(X, centers = 2, nstart = 20)
table(true = class_labels, cluster = km2$cluster)
```

```
##      cluster
## true  1  2
##    1 20  0
##    2 20  0
##    3  0 20
```

```
# (e) K=4
km4 <- kmeans(X, centers = 4, nstart = 20)
table(true = class_labels, cluster = km4$cluster)
```



```
##      cluster
## true  1  2  3  4
##      1  0  0 20  0
##      2  0  8  0 12
##      3 20  0  0  0
```

```
# (f) K-means on first two PCs
km_pc <- kmeans(pca_scores[,1:2], centers = 3, nstart = 20)
table(true = class_labels, cluster = km_pc$cluster)
```

```
##      cluster
## true  1  2  3
##      1 20  0  0
##      2  0  0 20
##      3  0 20  0
```

```
# (g) K-means on scaled data (already scaled? we used scale in PCA; now scale each variable to sd=1)
X_scaled <- scale(X)
km_scaled <- kmeans(X_scaled, centers = 3, nstart = 20)
table(true = class_labels, cluster = km_scaled$cluster)
```

```
##      cluster
## true  1  2  3
##      1 20  0  0
##      2  0 20  0
##      3  0  0 20
```

```
# Compare: scaling changes the relative importance of variables; here it may improve or worsen separation
# In this simulation, scaling was already used in PCA, but K-means on raw data vs scaled can differ.
cat("K-means on raw data (K=3) vs true:\n")
```

```
## K-means on raw data (K=3) vs true:
```

```
print(table(class_labels, km3$cluster))
```

```
##
## class_labels  1  2  3
##              1  0  0 20
##              2  0 20  0
##              3 20  0  0
```

```
cat("K-means on scaled data (K=3):\n")
```

```
## K-means on scaled data (K=3):
```

```
print(table(class_labels, km_scaled$cluster))
```

```
##
## class_labels  1  2  3
##              1 20  0  0
##              2  0 20  0
##              3  0  0 20
```